

Richard Smith (richard@metafoo.co.uk)
Andrew Sutton (andrew.n.sutton@gmail.com)
Daveed Vandevoorde (daveed@edg.com)

Immediate functions

Revisions

Revision 0: Original proposal with wording.

Revision 1: Introduce `constexpr!` token and allow it in lambda expressions.

Revision 2: Replace `constexpr!` by `constexpr`.

Revision 3: Address CWG review notes.

Introduction and motivation

The `constexpr` specifier applied to a function or member function indicates that a call to that function might be valid in a context requiring a constant-expression. It does not *require* that every such call be a constant-expression. Sometimes, however, we want to express that a function should *always* produce a constant when called (directly or indirectly), and a non-constant result should produce an error. Such a function is called an *immediate* function. We propose the *decl-specifier* `constexpr` for that purpose:

```
constexpr int sqr(int n) {  
    return n*n;  
}  
constexpr int r = sqr(100); // Okay.  
int x = 100;  
int r2 = sqr(x); // Error: Call does not produce a constant.
```

There is one exception to the rule that a call to a `constexpr` function must be a constant-expression: If the call appears directly in another `constexpr` function, it need not be a constant-expression at that point (since a call to the enclosing function would ultimately have to produce a core constant expression, we do not have to worry about ever having to evaluate that nested call at run time). This permits composition of `constexpr` functions:

```
constexpr int sqrsqr(int n) {  
    return sqr(sqr(n)); // Not a constant-expression at this point,  
}                        // but that's okay.  
  
constexpr int dblsqr(int n) {  
    return 2*sqr(n); // Error: Enclosing function is not  
}                  // constexpr.
```

Finally, as is the case with a bound member function (e.g., `"x.f"`), a `constexpr` function can only be called; no pointer or reference to it can be formed:

```
using Int2Int = int(int);  
Int2Int *pf = sqr; // Error.
```

One consequence of this specification is that an immediate function never needs to be seen by a back end. That means that immediate functions are an alternative to many function-style macros. It also means that, unlike plain `constexpr` functions, `constexpr` functions are unlikely to show up in symbolic debuggers. That runs into the broader issue that programmers currently have few tools to debug compile-time evaluation: This proposal does not aim at solving that problem in any way.

Variables

Given that `constexpr` functions are a useful alternative to function-style macros, it is natural to wonder whether we could make `constexpr` *variables* a useful alternative to non-function-style macros. We'd call such variables — which would never be seen by a back end — *immediate* variables:

```
constexpr int RN = 42;
int LN = 42;

int f(int const&);

int r1 = f(LN); // Direct reference binding.
int r2 = f(RN); // Not a direct binding (temporary created).
```

Unfortunately, the exact specification of such a facility raises some difficult questions, some of which relate to the topic of enabling class types for nontype template parameters as proposed in [P0732R1](#). This paper therefore does not pursue that extension at this time.

Notes

The desire for functionality like this comes up regularly. Sometimes, the motivation is to eliminate macros, and sometimes it just falls naturally out of the development process (e.g., helper functions meant to create nontype template argument values).

The impetus for the present paper, however, is the work being done by SG7 in the realm of compile-time reflection. There is now general agreement that future language support for reflection should use `constexpr` functions, but since "reflection functions" typically *have* to be evaluated at compile time, they will in fact likely be *immediate* functions.

The general principles of this facility were first discussed in Kona (2017) as part of the discussion of P0595r0. At the time, the following poll was recorded:

Provide a way to write a function that is guaranteed to be produce a constant expression, violations diagnosed at call site? (Direction poll for [immediate functions], ignoring syntax):
SF: 8 | F: 13 | N: 2 | A: 0 | SA: 0

(See <http://wiki.edg.com/bin/view/Wg21kona2017/P0595R0>. The syntax under consideration at that time was `do constexpr` instead of `constexpr`.)

At the Rapperswil meeting in 2018, essentially this proposal [was accepted](#) by EWG except the specifier `constexpr!` was used instead of `constexpr`. There was some discussion of syntax at the time, but the proposed spelling had reasonably strong support:

Proposal as presented for C++20
SF: 19 | F: 22 | N: 1 | A: 2 | SA: 0

Use "true constexpr" instead of "constexpr!"

However, reflector discussions later indicated an alternative would be preferred. This paper presents one such an alternative. Another that is also liked is `constfold` (if that happens to be preferred, a simple search-and-replace in the proposed wording below is all that is needed). Both `consteval` and `constfold` were searched for in Google's extensive codebase (which includes many open-source projects in addition to Google's own code) and no conflicts were found.

Acknowledgments

Thanks to Vittorio Romeo for catching a missing wording change.

Wording changes

Add the keyword `consteval` to Table 5 ("Keywords") in [lex.key].

Add to the grammar of [dcl.spec] paragraph 1:

decl-specifier:
 storage-class-specifier
 defining-type-specifier
 function-specifier
 friend
 typedef
 constexpr
 consteval
 inline

Update [expr.prim.lambda] paragraph 3 as follows:

In the *decl-specifier-seq* of the *lambda-declarator*, each *decl-specifier* shall ~~either~~ be one of mutable, ~~or~~ constexpr, or consteval. [Note: The trailing *requires-clause* is described in Clause 11. — end note]

Update [expr.prim.lambda.closure] paragraph 4 as follows:

... The function call operator or any given operator template specialization is a constexpr function if either the corresponding *lambda-expression*^{™s} *parameter-declaration-clause* is followed by constexpr or consteval, or it satisfies the requirements for a constexpr function (9.1.5). It is an immediate function (`_dcl constexpr_`) if the corresponding *lambda-expression*^{™s} *parameter-declaration-clause* is followed by consteval. ...

Update [expr.prim.lambda.closure] paragraph 7 as follows:

... F is a constexpr function if the function call operator is a constexpr function and is an immediate function if the function call operator is an immediate function.

Update [dcl constexpr] paragraph 8 as follows:

The constexpr and consteval specifiers have ~~has~~ no effect on the type of a constexpr function or a constexpr constructor.

Add a paragraph after paragraph 2 of [expr.prim.id]:

An *id-expression* that denotes an immediate function (`_dcl constexpr_`) shall appear as a subexpression of an immediate invocation or in an immediate function context (`_expr const_`).

Change in [expr.const] paragraph 6:

...

- ...
- if the value is of pointer type, it contains the address of an object with static storage duration, the address past the end of such an object (7.6.6), the address of a **non-immediate** function, or a null pointer value, and
- if the value is of pointer-to-member-function type, it does not designate an immediate function, and
- ...

...

An entity is a permitted result of a constant expression if it is an object with static storage duration that is either not a temporary object or is a temporary object whose value satisfies the above constraints, or it is a **non-immediate** function. *[Example:*

```
constexpr int f() { return 42; }
constexpr auto g() { return f; }
constexpr int h(int (*p)() = g()) { return p(); }
constexpr int r = h(); // OK
constexpr auto e = g(); // ill-formed: a pointer to an immediate function is
                        // not a permitted result of a constant expression
```

--end example]

In [expr.const] insert a new paragraph immediately preceding paragraph 8 (the paragraph that defines *potentially constant evaluated*) with the following context:

An expression or conversion is *in an immediate function context* if it is potentially evaluated and its innermost non-block scope is a function parameter scope of an immediate function. An expression or conversion is an *immediate invocation* if it is an explicit or implicit invocation of an immediate function and is not in an immediate function context. An immediate invocation shall be a constant expression. *[Note: An immediate invocation is evaluated even in an unevaluated operand. --end note]*

In [intro.execution] paragraph 5 add a bullet:

A *full-expression* is

- ...
- a *constant-expression* (7.7),
- an immediate invocation (`_expr.const_`)
- ...

In [decl.type.simple] paragraph 5 make the following change in the first sentence:

If the operand of a decltype-specifier is a prvalue and is not a (possibly parenthesized) immediate invocation (`_expr.const_`), the temporary...

Insert a bullet after the first bullet (8.1) of the definition of *potentially constant evaluated* in [expr.const] paragraph 8:

An expression or conversion is *potentially constant evaluated* if it is:

- ...

- an immediate invocation,
- ...

...

and add an example at the end of the paragraph:

[*Example:*

```
struct N {
    constexpr N() {}
    N(N const&) = delete;
};
template<typename T> constexpr void bad_assert_copyable() { T t; T t2 = t; }
using ineffective = decltype(bad_assert_copyable<N>());
// bad_assert_copyable<N> is not needed for constant evaluation
// (and thus not instantiated)
template<typename T> consteval void assert_copyable() { T t; T t2 = t; }
using check = decltype(assert_copyable<N>());
// error: assert_copyable<N> is instantiated (because it is needed for constant
// evaluation), but the attempt to copy t is ill-formed
```

— end example]

Append to [dcl.spec] paragraph 2:

The `constexpr` and `consteval` *decl-specifiers* shall not both appear in a *decl-specifier-seq*.

Update [dcl.constexpr] paragraph 1 as follows:

The `constexpr` specifier shall be applied only to the definition of a variable or variable template or the declaration of a function or function template. The `consteval` specifier shall be applied only to the declaration of a function or function template. A function or static data member declared with the `constexpr` or `consteval` specifier is implicitly an inline function or variable (10.1.6). If any declaration of a function or function template has a `constexpr` or `consteval` specifier, then all its declarations shall contain ~~the `constexpr`~~ that same specifier. [*Note:* An explicit specialization can differ from the template declaration with respect to the `constexpr` or `consteval` specifier. — end note] [*Note:* Function parameters cannot be declared `constexpr`. — end note]

Update [dcl.constexpr] paragraph 2 as follows:

A `constexpr` or `consteval` specifier used in the declaration of a function that is not a constructor declares that function to be a *constexpr function*. Similarly, a `constexpr` or `consteval` specifier used in a constructor declaration declares that constructor to be a *constexpr constructor*. A function or constructor declared with the `consteval` specifier is called an *immediate function*. A destructor, an allocation function, or a deallocation function shall not be declared with the `consteval` specifier.

Update [dcl.fct.def.default] paragraph 3 as follows:

An explicitly-defaulted function that is not defined as deleted may be declared `constexpr` or `consteval` only if it would have been implicitly declared as `constexpr`. [...]

Add a footnote to [class.virtual] paragraph 1:

... A class that declares or inherits a virtual function is called a polymorphic class. [Footnote: If all virtual functions are immediate functions, the class is still polymorphic even though its internal representation might not otherwise require any additions for that polymorphic behavior.]

Add the following paragraph following [class.virtual] paragraph 17:

A `constexpr` virtual function shall not override a virtual function that is not `constexpr`. A `constexpr` virtual function shall not be overridden by a virtual function that is not `constexpr`.

Update [temp.explicit] paragraph 1 as follows:

[...]An explicit instantiation of a function template, member function of a class template, or variable template shall not use the `inline`~~or~~, `constexpr`, or `constexpr` specifiers.

Change [diff.cpp17.lex] paragraph 1 as follows:

...

Rationale: ... The `constexpr` keyword is added to declare immediate functions (`_dcl.constexpr`).

Effect on original feature: Valid ISO C++ 2017 code using `concept`, `constexpr`, or requires as an identifier is not valid in this International Standard.